

AIKernel Hash-Anchored Trust Layer (HATL): A Hybrid Symmetric Ledger with Hash-Based Public Anchors

Symmetric Ratcheted Micro-Ledgers for Securing Stochastic AI Inference Tracks

Takuya Sogawa

2026-06-02

AIKernel Hash-Anchored Trust Layer (HATL): A Hybrid Symmetric Ledger with Hash-Based Public Anchors

Author: Takuya Sogawa

Affiliation: AIKernel Project

ORCID: <https://orcid.org/0009-0009-7499-2595>

DOI: <https://doi.org/10.5281/zenodo.20502685>

Version: v1.0

Document license: CC-BY-4.0

Code specification license: Apache-2.0

Abstract

Autonomous AI systems increasingly execute high-frequency inference calls, state transitions, retrieval operations, tool invocations, and capability updates. For such systems, the execution record is not merely an application log: it is the audit substrate from which reproducibility, accountability, incident analysis, and roll-back decisions are derived. A Semantic Context Operating System such as AIKernel therefore requires a trust layer capable of preserving inference tracks and ReplayLogs with deterministic integrity while remaining fast enough for runtime use.

This paper introduces the **AIKernel Hash-Anchored Trust Layer (HATL)**, a hybrid trust architecture that separates high-frequency local commitment from lower-frequency public anchoring. The inner layer is a symmetric, ratcheted micro-ledger built from HMAC-SHA-512, HKDF, hash chaining, and Merkle accumulation. The outer layer periodically commits compact state roots through hash-based public anchors, using stateful hash-based signature families such as LMS/XMSS where strict state management is available, or stateless SLH-DSA where operational simplicity is preferred.

HATL is designed to avoid placing heavyweight public-key signatures on every inference step. Instead, it commits each runtime event into a forward-ratcheted local ledger and publishes batched roots at policy-defined intervals. This gives the runtime low-latency local integrity, while giving auditors a compact public verification path. The paper also defines Digital Deed identity records, quorum-based revocation governance, repository specifications, and implementation discipline for side-channel-aware runtime deployment.

The contribution of this paper is architectural rather than a claim of a new cryptographic primitive. HATL composes established primitives into a layered trust substrate for Semantic Context Operating Systems. Its security claims are conditional on the standard assumptions of the underlying primitives, correct key lifecycle management, correct canonicalization, and appropriate public-ledger finality policies.

Keywords

AIKernel; Hash-Anchored Trust Layer; HATL; Hash-Based Signatures; Symmetric Ledger; HMAC; HKDF; Merkle Tree; ReplayLog; Digital Deed; Semantic Context Operating System; AI Governance; Forward Secrecy; Public Anchoring; Post-Quantum Transition.

1. Introduction

Large Language Model (LLM)-based systems increasingly operate as autonomous or semi-autonomous software components. They read context, generate structured plans, invoke tools, update task state, and interact with external systems. In AIKernel terminology, such behavior produces an inference track: a sequence of input contexts, intermediate decisions, outputs, tool calls, capability checks, and execution outcomes.

For a Semantic Context Operating System, the inference track must be replayable and auditable. If a tool invocation was allowed, the system must later be able to prove what context existed, which capability was active, which policy was applied, and which transition was committed. This requires more than ordinary logging. The log must be tamper-evident, causally ordered, deterministic under replay, and resilient against partial compromise.

A direct approach would be to sign every event using a public-key signature scheme. However, high-frequency inference systems can produce many small events per second. Signing every internal step with heavyweight public-key operations, including post-quantum signatures, may impose avoidable latency and storage overhead. At the same time, using only symmetric authentication is insufficient for public auditability, because an external verifier cannot independently validate a symmetric MAC without access to secret material.

HATL addresses this tension through a split-domain design:

1. **Inner symmetric domain:** high-frequency local commitments are authenticated by HMAC and evolved through HKDF-based key ratcheting.
2. **Accumulator domain:** local blocks are aggregated into Merkle roots over bounded anchor windows.
3. **Public anchor domain:** Merkle roots are periodically signed using hash-based public signatures and optionally published to a public or consortium ledger.
4. **Governance domain:** identity status, revocation state, and Digital Deed lifecycle are managed through quorum-based rules.

This design is motivated by a simple principle: runtime integrity and public auditability do not need to operate at the same frequency. HATL allows runtime events to be committed at the speed of symmetric cryptography, while public verifiability is applied to compact state roots.

1.1 Contributions

This paper makes the following contributions:

- **Hybrid trust-layer architecture.** It defines HATL as a split trust model combining an inner symmetric micro-ledger and an outer hash-based public anchoring layer.
- **Ratcheted symmetric micro-ledger.** It specifies a BlockMAC structure in which each event is chained to the previous commitment and the active key evolves forward using HKDF.
- **Hash-based public anchoring.** It defines an interval-based Merkle root publication model using LMS/XMSS or SLH-DSA as public anchors.
- **Digital Deed governance.** It introduces Digital Deed identity records and quorum-based revocation semantics for compromised or policy-invalid nodes.
- **Repository and implementation discipline.** It specifies a reproducible repository layout, interface sketches, schema definitions, and security hardening rules.
- **Explicit non-claims.** It states the limits of the model to avoid overstating formal proof, post-quantum guarantees, or performance results beyond available evidence.

1.2 Scope and Non-Claims

HATL is a composition architecture, not a new cryptographic primitive. This paper does not claim:

1. that HATL replaces formally analyzed authenticated data structures;
2. that any blockchain provides absolute finality under all adversarial conditions;
3. that HMAC, HKDF, LMS, XMSS, or SLH-DSA are newly invented by this paper;
4. that stateful HBS can be safely used without strict one-time-signature state management;
5. that the prototype throughput figures should be treated as universal benchmarks;
6. that anchoring hashes on a public ledger automatically satisfies every legal or regulatory requirement.

The intended claim is narrower: HATL offers a practical architectural pattern for coupling fast local symmetric commitments with slower but publicly verifiable hash-based anchors.

2. Background and Related Work

2.1 HMAC and HKDF

HMAC is a keyed message authentication construction based on cryptographic hash functions. In HATL, HMAC-SHA-512 is used to authenticate local ledger blocks. HKDF is used to derive forward-evolved epoch keys from prior key material and contextual execution hashes. The design does not require exposing symmetric keys to public verifiers; instead, public auditability is supplied through Merkle roots and public anchors.

2.2 Hash-Based Signatures

Hash-based signatures are attractive for long-term trust anchors because their security relies primarily on properties of hash functions rather than algebraic assumptions such as discrete logarithms or lattice hardness. HATL treats two families differently:

- **Stateful HBS:** LMS and XMSS can be efficient and conservative, but they require strict state tracking to prevent one-time-signature reuse.
- **Stateless HBS:** SLH-DSA avoids persistent signing-state hazards at the cost of larger signatures and different performance tradeoffs.

HATL therefore does not mandate a single public anchor mechanism. The anchor strategy is a policy choice, with stateful HBS appropriate for controlled infrastructures and stateless HBS appropriate when operational simplicity is more important than size.

2.3 Merkle Accumulation

A Merkle tree compresses a sequence of event commitments into a single root. HATL uses Merkle accumulation to avoid publishing every runtime event to a public ledger. Only the root of an anchor window is signed and published. This reduces public storage and fee pressure while preserving an inclusion proof path for individual events.

2.4 Semantic Context Operating Systems

AIKernel frames AI execution as an operating-system-like process: context is governed memory, tools are capabilities, execution is policy-mediated, and ReplayLogs are audit objects. HATL adds a cryptographic trust layer to this model by making the execution track tamper-evident and externally anchorable.

3. Threat Model and Security Goals

3.1 Adversary Model

We consider an adversary with the following capabilities:

- **Remote input manipulation:** prompt injection, malformed material insertion, or attempted capability escalation.
- **Storage extraction:** compromise of local databases, ROM snapshots, cache directories, or archived ReplayLogs.
- **Runtime key exposure:** temporary exposure of the currently active symmetric epoch key.
- **Ledger-layer interference:** delayed finality, chain reorganization, anchor censorship, or invalid anchor replay.
- **Governance manipulation:** fraudulent revocation proposals or insufficient quorum submissions.

The model allows a strong future adversary, including quantum-accelerated search capabilities. However, the paper avoids claiming unconditional security. The security properties remain conditional on the assumed strength of the selected hash functions, correct key management, and correct implementation of the anchor policy.

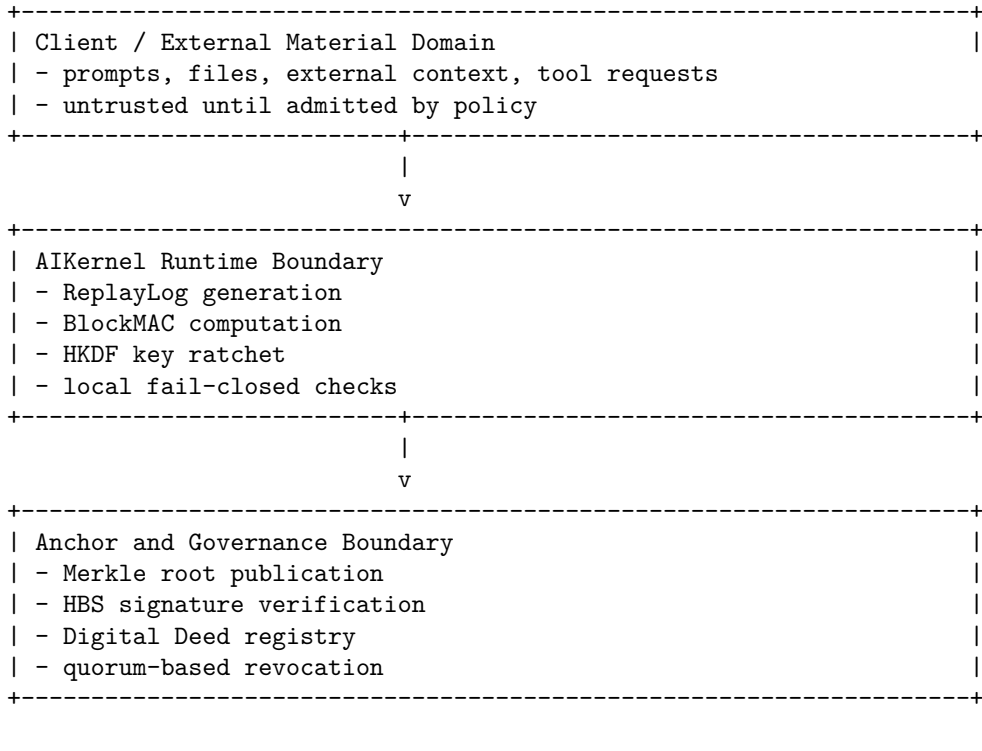
3.2 Security Objectives

HATL targets the following objectives:

- **Forward secrecy for local keys:** compromise of the current epoch key should not enable derivation of prior epoch keys when the ratchet is correctly implemented.

- **Tamper evidence:** modification of a committed event should break the local chain, the Merkle root, or the public anchor verification path.
- **Bounded public auditability:** external auditors should be able to verify that a provided event belongs to a published anchor window without receiving symmetric key material.
- **Revocation enforceability:** compromised Digital Deeds should be removable from the valid execution set by quorum-governed policy.
- **Fail-closed operation:** ambiguous or invalid trust states should halt privileged execution rather than silently accepting unverified history.

3.3 Trust Boundaries



4. HATL Architecture

4.1 Runtime Flow

HATL maps an inference execution track into a cryptographic commitment sequence:

Inference Event

- > canonical event representation
- > event hash
- > BlockMAC using current epoch key
- > HKDF ratchet to next epoch key
- > Merkle leaf accumulation
- > periodic root anchoring
- > public anchor verification

The inner loop is symmetric and fast. The outer loop is public and slower. This separation is the central performance and governance idea of HATL.

4.2 BlockMAC Record

A BlockMAC record commits to an event, its causal predecessor, and the current policy context.

```

{
  "sequence_number": 1024,
  "epoch_id": 7,
  "previous_block_mac": "hex-encoded-64-byte-value",

```

```

    "event_hash": "hex-encoded-64-byte-value",
    "execution_outcome_hash": "hex-encoded-64-byte-value",
    "capability_scope_hash": "hex-encoded-64-byte-value",
    "hmac_signature": "hex-encoded-64-byte-value"
}

```

The message authenticated by HMAC should be a deterministic canonical serialization of all fields except `hmac_signature`. JSON Canonicalization Scheme (JCS) or an equivalent deterministic binary format should be used to prevent cross-platform ambiguity.

4.3 Key Ratchet

Let K_t be the active epoch key at sequence t . Let E_t be the canonical event hash or execution outcome hash. The next key is derived as:

$$K_{t+1} = HKDF_Expand(HKDF_Extract(salt_t, K_t), info = E_t \parallel t, L = 64)$$

A valid implementation must erase or make inaccessible K_t after K_{t+1} is derived and the relevant BlockMAC has been computed. If old keys are retained, the forward-secrecy property is weakened or lost. In managed runtimes such as C#/.NET, this requirement is an engineering discipline rather than an automatic property of garbage collection: implementations should avoid ordinary long-lived byte arrays for secrets and should use bounded buffers such as `Span<byte>/ReadOnlySpan<byte>`, pinned or native memory where appropriate, and explicit clearing through APIs such as `CryptographicOperations.ZeroMemory`.

4.4 Merkle Anchor Window

Let an anchor window contain BlockMAC values $B_i \dots B_j$. Each leaf is computed as:

$$L_k = H("HATL - LEAF" \parallel canonical(B_k))$$

Internal nodes are computed as:

$$N_{parent} = H("HATL - NODE" \parallel N_{left} \parallel N_{right})$$

Domain separation labels are recommended to prevent accidental ambiguity between leaf and internal node hashing. At the end of an anchor window, the root is embedded in a public anchor document.

4.5 Public Anchor Document

```

{
  "anchor_id": 42,
  "ledger_identity": "AIKernel-HATL-demo-ledger",
  "policy_epoch": 3,
  "batch_start_seq": 1000,
  "batch_end_seq": 1999,
  "accumulated_merkle_root": "hex-encoded-root",
  "previous_anchor_hash": "hex-encoded-anchor-hash",
  "hbs_signature_type": "SLH-DSA",
  "hbs_signature_raw": "hex-or-base64-signature"
}

```

The public anchor document binds the batch root to the previous anchor hash. This creates a public, append-only anchor chain. The actual event records may remain private while still being provable through inclusion paths.

4.6 Digital Deed Identity

A Digital Deed is the identity and capability anchor of a node.

```

{
  "node_uuid": "3ec7fe55-9b89-4a74-9a92-cf68e9ad1a6d",
  "rom_identity_commitment": "hex-encoded-rom-commitment",
  "assigned_public_anchor_key": "encoded-public-key",
  "capability_contract_hash": "hex-encoded-capability-contract",
  "status": "ACTIVE",
  "policy_epoch": 3
}

```

Digital Deeds are not merely certificates. They are governance objects that can transition between **ACTIVE**, **SUSPENDED**, and **REVOKED** states.

5. Governance and Revocation

5.1 Quorum-Governed Revocation

A compromised node should not remain trusted simply because its earlier anchors were valid. HATL therefore separates historical verification from identity status. A public anchor may remain historically valid, while the associated Digital Deed may later become revoked.

Algorithm: EvaluateRevocationProposal

Input: proposal $P(\text{target_node}, \text{evidence_hash}, \text{quorum_signatures})$

Output: updated Digital Deed status or rejection

1. If P contains fewer than m valid governance signatures, reject.
2. Verify each governance signature against the configured policy epoch.
3. Verify that `evidence_hash` is bound to a valid audit artifact or incident record.
4. If signature validation, evidence validation, policy lookup, or network-dependent quorum resolution returns `Indeterminate`, reject the proposal and fail closed.
5. If verification succeeds, set `DigitalDeed(target_node).status = REVOKED`.
6. Broadcast the revocation event to runtime policy enforcement points.
7. Runtime nodes must fail closed for new privileged work from the revoked identity.

5.2 Challenge Window

For operational safety, HATL may include a challenge window. During this window, a node or auditor can present an alternative audit path that disputes the evidence behind a suspension or revocation. The challenge mechanism is a governance policy, not a cryptographic primitive.

5.3 Policy Epochs

All governance decisions are bound to policy epochs. A policy epoch identifies the active rule set, quorum threshold, validator membership, anchor algorithm, and canonicalization version. This prevents ambiguous verification under changing governance rules.

6. Security Analysis

6.1 Forward Ratchet Claim

Claim. If HKDF is instantiated with a secure hash function, and if old epoch keys are erased after ratcheting, then compromise of K_T does not directly reveal K_t for $t < T$.

Sketch. HATL derives K_{t+1} from K_t through HKDF and event-bound context. Recovering K_t from K_{t+1} would require inverting the KDF construction or discovering useful preimage information about the internal hash-based extraction and expansion outputs. Under the standard assumptions of HKDF and the selected hash function, this is computationally infeasible for polynomial-time adversaries. The property depends on secure erasure of old keys; if prior keys remain in memory, crash dumps, swap files, diagnostic logs, or managed

heap remnants, the claim no longer holds. For C#/.NET deployments, forward secrecy therefore requires an explicit secret-handling policy, including bounded secret lifetimes, avoidance of unnecessary copying, pinned or native buffers for high-risk key material, and best-effort zeroization using `CryptographicOperations.ZeroMemory` before buffers are released.

6.2 Tamper Evidence Claim

Claim. If a committed event record is modified after anchoring, then the modification changes its BlockMAC, Merkle path, or anchor root unless the adversary finds a collision or forges the relevant authentication value.

Sketch. A committed event contributes to a BlockMAC. The BlockMAC contributes to a Merkle leaf. The leaf contributes to a root. The root is signed or otherwise publicly anchored. Changing a historical event changes the leaf and therefore the root, unless the attacker can produce a collision or forge the authentication chain. HATL therefore provides tamper evidence, not magical deletion resistance. Physical deletion can still occur, but unverifiable gaps should be detected by sequence and anchor checks.

6.3 Public Verifiability Claim

Claim. An external auditor can verify inclusion of a disclosed event in a public anchor without receiving local symmetric keys.

Sketch. The auditor receives the event record, its canonical serialization, the BlockMAC record or disclosed leaf, the Merkle path, and the public anchor. The auditor recomputes the inclusion path to the root and verifies the HBS signature or ledger commitment. The auditor does not need the symmetric key unless the audit policy requires verification of the HMAC itself. In privacy-preserving deployments, the public audit proves inclusion in an anchored batch, while local HMAC validation remains inside the trusted runtime boundary.

6.4 Assumptions and Failure Modes

HATL depends on the following assumptions:

1. chosen hash functions and HMAC/HKDF constructions remain secure for the deployment horizon;
2. canonicalization is deterministic and versioned;
3. stateful HBS counters are never reused;
4. keys are generated with sufficient entropy;
5. old symmetric keys are erased or made inaccessible;
6. public anchor finality policy is appropriate for the underlying ledger;
7. governance quorum keys are protected and rotated.

Failure of any assumption weakens the guarantees. The most severe engineering risks are stateful HBS index reuse, secret-key retention, non-deterministic serialization, and over-trusting public ledger finality.

7. Prototype and Evaluation Plan

7.1 Prototype Scope

The v1.0 package defines an architecture and repository specification. It does not include a fully certified production implementation. The reference repository profile in Part II defines expected source topology, schema files, contract specimens, and test scaffolding.

7.2 Metrics

A reproducible evaluation should report:

Metric	Meaning
BlockMAC throughput	number of local event commitments per second
Ratchet latency	time to derive the next epoch key
Merkle build cost	time to build roots for different window sizes
Anchor publication latency	time from root generation to public commitment
Verification latency	time to verify inclusion and public signature
Memory allocation profile	allocated bytes per event or per anchor window

Metric	Meaning
Revocation propagation time	time to enforce a Digital Deed status change

7.3 Preliminary Performance Profile

The original design note includes prototype-oriented throughput targets around tens of thousands of local commitments per second for the symmetric inner loop. In this cleaned v1.0 manuscript, such numbers are treated as **preliminary prototype targets** rather than universal claims. Future releases should include raw benchmark scripts, hardware profiles, compiler versions, runtime configuration, and reproducibility logs.

8. Deployment and Operational Considerations

8.1 Stateful HBS Lifecycle

If LMS or XMSS is used, state management is critical. The implementation must prevent reuse of one-time-signature leaves even across crashes, rollbacks, backups, and concurrent signer instances. A safe deployment should use hardware-backed monotonic counters, transactional storage, or isolated signer services.

8.2 SLH-DSA Alternative

SLH-DSA avoids stateful signer counters, simplifying operational safety. The tradeoff is larger signatures and potentially different performance characteristics. HATL treats SLH-DSA as a valid anchor profile, especially when stateful signing discipline cannot be guaranteed.

8.3 Privacy and Metadata Leakage

Publishing anchor roots can reveal timing and volume metadata. Mitigations include fixed-size anchor windows, randomized publication delay within policy bounds, null-event padding, and private or consortium ledgers for sensitive deployments. These mitigations reduce leakage but do not prove zero information disclosure. A future privacy-preserving anchor profile may incorporate zero-knowledge proofs, such as zk-SNARKs, so that a public verifier can check that a committed batch followed the required ledger-transition rules without learning the underlying Merkle path, event history, or internal ReplayLog contents.

8.4 Legal and Compliance Notes

HATL stores cryptographic commitments, not raw inference content, on the public anchor plane. This is privacy-friendly, but it does not automatically satisfy every jurisdictional requirement. Deployments must evaluate whether hashes, metadata, timestamps, or linkage patterns constitute personal data under local law.

9. Repository Specification Summary

Part II of this release contains technical appendices and repository specifications. At minimum, a HATL reference repository should include:

```
/AIKernel.Hat1
  LICENSE
  README.md
  HATL_SPEC.md
/src
  /AIKernel.Hat1.Core
  /AIKernel.Hat1.Anchor
/schemas
  hatl-block-mac.schema.json
  hatl-public-anchor.schema.json
  hatl-digital-deed.schema.json
/contracts
  HatlGovernance.sol
```



```
/tests
  RatchetTests.cs
  AnchorVerificationTests.cs
  BenchmarkProfiles.json
```

The code components should use Apache-2.0. The explanatory documents should use CC-BY-4.0.

10. Limitations and Future Work

HATL has several limitations.

1. It composes primitives but does not replace formal cryptographic proofs of those primitives.
2. It relies on correct implementation of canonicalization, key erasure, and HBS state management.
3. Public ledger finality is ledger-specific and may be probabilistic.
4. Privacy mitigations reduce but do not eliminate metadata leakage.
5. Throughput depends on hardware, implementation, batch size, and logging policy.
6. Quorum governance depends on the security and independence of governance keys.

Future work should include executable reference implementations, formal models for the BlockMAC state machine, property-based tests for canonicalization, reproducible benchmarks, SLH-DSA/LMS profile comparison, privacy analysis of anchor timing leakage, and zero-knowledge verification profiles for proving rule-compliant log processing without disclosing ledger internals.

11. Conclusion

The AIKernel Hash-Anchored Trust Layer (HATL) is a trust-layer architecture for securing high-frequency AI execution tracks without forcing every internal event through a public-key signature path. Its core idea is to split commitment frequency from public verification frequency: local events are committed through a fast symmetric ratchet and later anchored through compact hash-based public roots.

For Semantic Context Operating Systems, this design offers a practical bridge between runtime performance and auditability. HATL gives AIKernel a way to preserve ReplayLogs as tamper-evident causal records while supporting public verification, Digital Deed governance, and policy-driven revocation. Its guarantees remain conditional on standard cryptographic assumptions and careful engineering discipline, but the architecture provides a clear foundation for future AIKernel trust infrastructure.

References

- National Institute of Standards and Technology. (2015). *Secure Hash Standard (SHS)*. FIPS PUB 180-4. <https://doi.org/10.6028/NIST.FIPS.180-4>
- National Institute of Standards and Technology. (2008). *The Keyed-Hash Message Authentication Code (HMAC)*. FIPS PUB 198-1. <https://doi.org/10.6028/NIST.FIPS.198-1>
- National Institute of Standards and Technology. (2024). *Stateless Hash-Based Digital Signature Standard*. FIPS PUB 205. <https://doi.org/10.6028/NIST.FIPS.205>
- Cooper, D. A., Apon, D. C., Dang, Q. H., Davidson, M. S., Dworkin, M. J., & Miller, C. A. (2020). *Recommendation for Stateful Hash-Based Signature Schemes*. NIST SP 800-208. <https://doi.org/10.6028/NIST.SP.800-208>
- Krawczyk, H., & Eronen, P. (2010). *HMAC-based Extract-and-Expand Key Derivation Function (HKDF)*. RFC 5869. <https://doi.org/10.17487/RFC5869>
- Huelsing, A., Butin, D., Gazdag, S., Rijneveld, J., & Mohaisen, A. (2018). *XMSS: eXtended Merkle Signature Scheme*. RFC 8391. <https://doi.org/10.17487/RFC8391>
- McGrew, D., Curcio, M., & Fluhrer, S. (2019). *Leighton-Micali Hash-Based Signatures*. RFC 8554. <https://doi.org/10.17487/RFC8554>

- Rundgren, A., Jordan, B., & Erdtman, S. (2020). *JSON Canonicalization Scheme (JCS)*. RFC 8785. <https://doi.org/10.17487/RFC8785>
- Shamir, A. (1979). How to share a secret. *Communications of the ACM*, 22(11), 612-613. <https://doi.org/10.1145/359168.359176>
- Merkle, R. C. (1989). A certified digital signature. In G. Brassard (Ed.), *Advances in Cryptology - CRYPTO '89 Proceedings* (pp. 218-238). Springer.
- Sogawa, T. (2026). *Interface-Led Architecture (ILA): A Software Development Methodology for the AI Era, Validated by the AIKernel Execution Model*. Zenodo. <https://doi.org/10.5281/zenodo.20290614>
- Sogawa, T. (2026). *AIKernel Formal Foundations: Contract-Based Semantic Execution for Governed AI Systems*. Zenodo. <https://doi.org/10.5281/zenodo.20460322>